Python

What is Programming?

Programming is the process of giving instructions to a computer to perform specific tasks.

What is a Program?

A program is a set of instructions written in a programming language that a computer can execute.

What is Syntax?

Syntax refers to the rules and structure that define how to write valid code in a programming language.

What is a Compiler?

A compiler converts the entire program into machine code **before** running it.

Example languages: C, C++

What is an Interpreter?

An interpreter converts and executes the program **line by line**.

Example languages: Python, JavaScript

Input → Process → Output

Every program follows this sequence:

- **Input:** Data you give.
- **Process:** Logic / calculations.
- **Output:** Final result.

What is an Editor?

An editor is software where you write your programs.

Examples:

- VS Code
- PyCharm
- Jupyter Notebook
- IDLE

Introduction to Python

What is Python?

Python is a high-level, easy-to-learn, general-purpose programming language used for building a wide range of applications such as data science, web apps, automation, AI, machine learning, and more.

Key Features of Python

1. **Simple and Easy to Learn**
   Python's syntax is clean and close to English, making it beginner-friendly.
2. **Interpreted Language**
   Executes code line-by-line. Helps in debugging easily.
3. **Dynamically Typed**
   No need to declare variable types. Python decides the type at runtime.
   Example: `x = 10`, `x = "Hello"` is valid.
4. **High-Level Language**
   You don't need to manage memory or system-level details.
5. **Object-Oriented Programming Support**
   Supports classes, objects, encapsulation, inheritance, and polymorphism.
6. **Cross-Platform**
   Works on Windows, macOS, Linux without changing the code.
7. **Extensive Standard Library**
   Built-in modules such as `math`, `os`, `datetime`, etc., reduce coding effort.
8. **Large Collection of External Libraries**
   - For data science → NumPy, Pandas, Matplotlib
   - For AI & ML → TensorFlow, Scikit-learn, PyTorch
   - For web → Django, Flask
9. **Supports Multiple Programming Paradigms**
   - Procedural programming
   - Object-oriented programming
   - Functional programming
10. **Open Source and Free**
    Anyone can download, use, and modify Python.
11. **Portable**
    Write once → run anywhere.

Basic Syntax

- Python files end with `.py`.
- Code runs line by line.
- a = 10
- b=20
- print(a+b)

Input and Output in Python

Input in Python

`input()` is used to take input from the user. It always returns the input as a **string**, so type conversion is sometimes needed.

```
variable = input("Enter something: ")

a = int(input("Enter a number: "))

b = int(input("Enter another number: "))

print(a + b)
```

Output in Python

`print()` is used to display messages or values on the screen.

```
print("Hello World")

print("Sum is", 10 + 20)
```

Variables and Identifiers

What is a Variable?

A variable is a name that stores a value in memory. It acts like a container where you store information.

What is an Identifier?

An identifier is the name you give to variables, functions, classes, etc. It is simply **a name used to identify something in Python**.

Example identifiers:

- my_age
- totalMarks
- sum1
- price_of_item

Rules for Identifiers

Can contain alphabets (A–Z, a–z), digits (0–9), and underscores (_).

Cannot start with a digit.

Cannot use special characters like @, #, $, %, ^, &, *, etc.

Cannot contain spaces.

Cannot use Python keywords (for, while, class, return, etc.).

Identifiers are case-sensitive (`age` ≠ `Age`).

Should be meaningful (recommended).

No length limit, but keep names short and clear.

Underscore (_) can be used at the beginning (e.g., `_value`).

What are Keywords?

Keywords are **reserved words** in Python that have a special meaning and purpose. You **cannot** use them as variable names, function names, or identifiers.

`if,else,elif,for,while,print,input`

What Are Data Types?

Data types define the type of data a variable can store. They determine:

- What values a variable can hold
- What operations can be performed on it

Python Data Types Table

Data Type	Category	Description	Example
int	Numeric	Whole numbers	`10`, `-5`
float	Numeric	Decimal numbers	`3.14`, `10.5`
str	Text	Sequence of characters	`"Hello"`, `'Python'`
bool	Boolean	True/False values	`True`, `False`
list	Collection	Ordered, changeable, allows duplicates	`[1, 2, 3]`
tuple	Collection	Ordered, unchangeable, allows duplicates	`(1, 2, 3)`
set	Collection	Unordered, no duplicates	`{1, 2, 3}`
dict	Mapping	Key-value pairs	`{"name": "Lx"}`
NoneType	Special	Represents no value	`None`

What is a String?

A **string** is a sequence of characters enclosed in single (`' '`), double (`" "`), or triple quotes.

String Methods

String methods are built-in functions used to perform operations on strings.

Python String Functions Table

Function	Description	Example Code
`upper()`	Converts entire string to uppercase	`"hello".upper()` → `HELLO`
`lower()`	Converts entire string to lowercase	`"HELLO".lower()` → `hello`
`title()`	Converts first letter of each word to capital	`"python programming".title()` → `Python Programming`
`capitalize()`	Capitalizes the first character of the string	`"python".capitalize()` → `Python`
`strip()`	Removes spaces from both ends	`" hello ".strip()` → `hello`
`lstrip()`	Removes spaces from the left side only	`" hello".lstrip()` → `hello`
`rstrip()`	Removes spaces from right side only	`"hello ".rstrip()` → `hello`
`replace(old, new)`	Replaces part of the string	`"cat".replace("c", "b")` → `bat`

`split(separator)`	Splits string into a list	`"a,b,c".split(",")` → `['a','b','c']`
`join()`	Joins list elements into string	`"-".join(['a','b','c'])` → `a-b-c`
`find()`	Returns index of first occurrence	`"hello".find("e")` → `1`
`rfind()`	Returns last occurrence index	`"hello".rfind("l")` → `3`
`index()`	Returns index (error if not found)	`"hello".index("l")` → `2`
`count()`	Counts occurrences of substring	`"hello".count("l")` → `2`
`startswith()`	Returns True if string starts with substring	`"hello".startswith("he")` → `True`
`endswith()`	Returns True if string ends with substring	`"hello".endswith("lo")` → `True`
`isdigit()`	Checks if all characters are digits	`"123".isdigit()` → `True`
`isalpha()`	Checks if all characters are alphabets	`"abc".isalpha()` → `True`

`isalnum()`	Checks for alphanumeric characters	`"abc123".isalnum()` → `True`
`islower()`	True if all characters are lowercase	`"hello".islower()` → `True`
`isupper()`	True if all characters are uppercase	`"HELLO".isupper()` → `True`
`swapcase()`	Swaps case	`"HeLLo".swapcase()` → `hEllO`
`zfill(width)`	Pads string with leading zeros	`"5".zfill(3)` → `005`

Indexing and Negative Indexing

What is Indexing?

Indexing means accessing individual characters or elements in a string (or list, tuple, etc.) using their **position number**.

- Indexing starts from **0** for the first character.

```
text = "Python"
print(text[0]) # P
print(text[1]) # y
print(text[5]) # n
```

What is Negative Indexing?

Negative indexing means accessing characters from **the end** of the string.

- Last character → index **-1**
- Second last → index **-2**

```
text = "Python"
print(text[-1]) # n
print(text[-2]) # o
print(text[-6]) # P
```

String Slicing

String slicing is used to extract a part of a string.

String [start:end:step]

```
text = "Python"

print(text[0:2]) # Py

print(text[1:4]) # yth

print(text[:3]) # Pyt

print(text[3:]) # hon

print(text[-1]) # n

print(text[-3:]) # hon

print(text[::2]) # Pto

print(text[::-1]) # Reverse -> nohtyP
```

Tuple in Python

A **tuple** is an ordered collection of items, similar to a list, but **immutable** (cannot be changed after creation).Allows duplicates.

```
my_tuple = (100, 250, 30,100)
```

Tuple Methods

No.	Method / Function	Description	Example
1	`count(value)`	Counts how many times a value appears	`(1,2,2,3).count(2)` → `2`
2	`index(value)`	Returns index of first occurrence	`(10,20,30).index(20)` → `1`
3	`len()`	Returns length of tuple	`len((1,2,3))` → `3`
4	`max()`	Returns largest value	`max((10,50,30))` → `50`

5	`min()`	Returns smallest value	`min((10,50,30))` → `10`
6	`sum()`	Returns sum of elements (if numeric)	`sum((1,2,3))` → `6`
7	`sorted()`	Returns a sorted list of tuple elements	`sorted((3,1,2))` → `[1,2,3]`
8	`tuple()`	Converts another datatype to tuple	`tuple([1,2])` → `(1,2)`

Mutable vs Immutable Data Types

Immutable Data Types

These **cannot be changed** after creation.

Examples: `int`, `float`, `str`, `tuple`, `bool`

```
x = "Hello"

x[0] = "J"     #Error
```

Mutable Data Types

These **can be changed** after creation.

Examples: `list`, `set`, `dict`

```
list1 = [1, 2, 3]

list1[0] = 10
```

Examples of Mutable and Immutable

INT

x = 10

Trying to modify a specific digit is impossible

x[0] = 5 Error: int is immutable

Instead, reassignment creates a NEW object

x = x + 5

FLOAT

```
pi = 3.14

# pi[0] = 9  Error: float is immutable

# Reassigning creates a new float object

pi = pi + 1.0
```

STRING

```
name = "Hello"
# name[0] = "J"  Error: strings cannot be changed

# Must create a NEW string
name = "J" + name[1:]
```

TUPLE

```
tup = (1, 2, 3)
# tup[0] = 10  Error: cannot modify tuple items

# Must create a new tuple if you want changes
new_tup = (10,) + tup[1:]
```

LIST - MUTABLE

```
lst = [1, 2, 3]
lst[0] = 10         # Allowed – list item changed
lst.append(4)
```

SET - MUTABLE

```
s = {1, 2, 3}
s.add(4)            # Allowed – adds new value
s.remove(2)
```

DICTIONARY - MUTABLE

```
info = {"name": "ABC", "age": 20}

info["age"] = 30        #Allowed – updating value

info["city"] = "Kochi"
```

List Creation in Python

A **list** is an ordered, mutable collection of items.

```
my_list = [10, 20, 30]
```

1. Empty list

```
lst = []
```

2. List with values

```
lst = ["apple", "banana", "mango"]
```

3. Using list() function

```
lst = list((1, 2, 3))
```

4. List with mixed data types

```
lst = [10, "Lx", 3.14, True]
```

5. Nested lists

```
lst = [[1, 2], [3, 4]]
```

List Methods

Lists are **mutable**, meaning you can modify them anytime. Below is a complete table of commonly used list methods.

Method	Description	Example Code
`append(x)`	Adds an item at the end	`lst.append(10)`
`insert(i, x)`	Inserts item at index `i`	`lst.insert(1, 50)`
`extend(iterable)`	Adds multiple items to the list	`lst.extend([4, 5])`

`remove(x)`	Removes the first occurrence of value `x`	`lst.remove(20)`
`pop(i)`	Removes item at index `i` (default last)	`lst.pop(0)`
`clear()`	Removes all items from the list	`lst.clear()`
`sort()`	Sorts the list in ascending order	`lst.sort()`
`reverse()`	Reverses the list	`lst.reverse()`
`count(x)`	Counts occurrences of `x`	`lst.count(2)`
`index(x)`	Returns index of value `x`	`lst.index(50)`

List Slicing

List slicing allows extracting specific parts of a list using index positions.

Syntax: list[start:end:step]

Example List

lst = [10, 20, 30, 40, 50]

1. Basic Slicing

print(lst[1:4]) # [20, 30, 40]

2. Omitting Start or End

print(lst[:3]) # [10, 20, 30]

print(lst[2:]) # [30, 40, 50]

3. Negative Indexing

```
print(lst[-3:])   # [30, 40, 50]

print(lst[:-2])   # [10, 20, 30]
```

4. Step Value

```
print(lst[::2])   # [10, 30, 50]

print(lst[::-1])  # [50, 40, 30, 20, 10]
```

Dictionary in Python

A **dictionary** is a collection of key–value pairs.

- Ordered (Python 3.7+)
- Mutable
- Keys must be unique
- Values can be any data type

```
student = {"name": "Alice", "age": 25, "city": "Kochi"}
```

Dictionary Methods

Method	Description	Example Code
`get(key)`	Returns value of the specified key	`d.get("name")`
`keys()`	Returns all keys in the dictionary	`d.keys()`
`values()`	Returns all values in the dictionary	`d.values()`
`items()`	Returns list of (key, value) pairs	`d.items()`

`update({})`	Updates dictionary with new key-value pairs	`d.update({"age": 30})`
`pop(key)`	Removes item with given key	`d.pop("age")`
`popitem()`	Removes last inserted item	`d.popitem()`
`clear()`	Removes all items from dictionary	`d.clear()`
`setdefault(key, default)`	Returns value; inserts key with default if missing	`d.setdefault("city", "Kochi")`
`fromkeys(iterable, value)`	Creates dictionary with given keys and same value	`dict.fromkeys([1,2,3], 0)`

Set in Python

A **set** is an unordered collection of **unique** elements.

- Unordered
- Mutable
- No duplicates

set1 = {1, 2, 3, 4, 5}

Set Methods

Method	Description	Example Code
`add(x)`	Adds an element to the set	`s.add(10)`

`remove(x)`	Removes element (error if missing)	`s.remove(3)`
`discard(x)`	Removes element (no error if missing)	`s.discard(5)`
`pop()`	Removes and returns a random element	`s.pop()`
`clear()`	Removes all elements	`s.clear()`
`update(iterable)`	Adds multiple elements	`s.update([4,5,6])`
`union(other)`	Returns union of sets	`s.union(s2)`
`intersection(other)`	Returns common elements	`s.intersection(s2)`
`difference(other)`	Elements in s but not in s2	`s.difference(s2)`
`symmetric_difference(other)`	Elements not common in sets	`s.symmetric_difference(s2)`
`issubset(other)`	Checks if set is subset	`s.issubset(s2)`
`issuperset(other)`	Checks if set is superset	`s.issuperset(s2)`
`copy()`	Returns a shallow copy of set	`s.copy()`

Frozenset

A **frozenset** is an immutable set.

- Cannot add, remove, or modify elements
- Useful when needing a set that should not change

fs = frozenset([1, 2, 3])

print(fs)

fs = frozenset([1, 2, 3])

fs.add(4) → Error: cannot modify frozenset

Method	Description	Example Code
`copy()`	Returns a shallow copy of the frozenset	`fs.copy()`
`union(other)`	Returns union of sets	`fs.union(fs2)`
`intersection(other)`	Common elements	`fs.intersection(fs2)`
`difference(other)`	Items in fs not in fs2	`fs.difference(fs2)`
`symmetric_difference(other)`	Elements not common in both	`fs.symmetric_difference(fs2)`
`issubset(other)`	Checks if fs is a subset	`fs.issubset(fs2)`
`issuperset(other)`	Checks if fs is a superset	`fs.issuperset(fs2)`

`isdisjoint(other)`	True if sets have no common items	`fs.isdisjoint(fs2)`

Operators in Python

1. Arithmetic Operators

Operator	Meaning	Example
`+`	Addition	`10 + 5 = 15`
`-`	Subtraction	`10 - 5 = 5`
`*`	Multiplication	`5 * 3 = 15`
`/`	Division	`10 / 4 = 2.5`
`//`	Floor division	`10 // 4 = 2`
`%`	Modulus (remainder)	`10 % 4 = 2`
`**`	Exponent (power)	`2 ** 3 = 8`

2. Relational / Comparison Operators

Operator	Meaning	Example
`==`	equal to	`5 == 5` → True
`!=`	not equal to	`5 != 3` → True
`>`	greater than	`10 > 5`
`<`	less than	`3 < 9`
`>=`	greater or equal	`5 >= 5`
`<=`	less or equal	`4 <= 6`

3. Logical Operators

Operator	Meaning	Example
`and`	True if both conditions are True	`5 > 2 and 3 < 4` → True
`or`	True if any condition is True	`5 > 10 or 3 < 4` → True
`not`	Reverses result	`not(True)` → False

4. Assignment Operators

Operator	Meaning	Example
`=`	Assign	`x = 5`
`+=`	Add & assign	`x += 2`
`-=`	Subtract & assign	`x -= 3`
`*=`	Multiply & assign	`x *= 4`
`/=`	Divide & assign	`x /= 2`

5. Membership Operators

Operator	Meaning	Example
`in`	checks membership	`"a" in "apple"` → True
`not in`	checks absence	`3 not in [1,2]` → True

6. Identity Operators

Operator	Meaning	Example
`is`	same identity	`x is y`
`is not`	not same identity	`x is not y`

Conditional statements

1. if Statement

```
age = 20

if age >= 18:
    print("Eligible to vote")
```

2. if–else Statement

```
age = 15

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

3. if–elif–else Statement

```
mark = 85

if mark >= 90:
    print("A Grade")
elif mark >= 75:
    print("B Grade")
elif mark >= 60:
    print("C Grade")
else:
    print("Fail")
```

Control Statements

While Loop

A **`while` loop** executes a block of code **repeatedly as long as a condition is True**.

```
while condition:
    Statements
```

Example

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

WHILE + ELSE

The `else` block runs only when the loop finishes normally (not when it is stopped by `break`).

Syntax:

```
while condition:
    statements
else:
    statements
```

Example

```
i = 1

while i <= 3:
    print(i)
    i += 1
else:
    print("Loop finished")
```

WHILE + BREAK

break is used to stop the loop immediately, even if the condition is still True.

```
i = 1

while i <= 10:
    if i == 5:
        break
    print(i)
    i += 1
```

WHILE + CONTINUE

`continue` **skips the current loop iteration** and jumps to the next round.

```
i = 0

while i < 5:

    i += 1

    if i == 3:

        continue

    print(i)
```

range()

`range()` is used to generate a sequence of numbers.Mostly used with `for` loops.

```
range(stop)            # 0 to stop-1

range(start, stop)     # start to stop-1

range(start, stop, step)


print(list(range(5)))          # [0, 1, 2, 3, 4]

print(list(range(2, 6)))       # [2, 3, 4, 5]

print(list(range(1, 10, 2)))   # [1, 3, 5, 7, 9]
```

for loop

Used to **iterate over a sequence** (list, string, range, etc.).

```
for i in range(1, 6):
    print(i)
```

```
fruits = ["apple", "banana", "mango"]

for f in fruits:

    print(f)
```

`for` + `else` - else runs only if the loop completes normally (no break).

```
for i in range(1, 4):

    print(i)

else:

    print("Loop finished")
```

break with for loop - Stop the loop immediately.

```
for i in range(1, 11):

    if i == 5:

        break

    print(i)
```

`continue` with `for` loop - Skips the current iteration and goes to the next one.

```
for i in range(1, 6):

    if i == 3:

        continue

    print(i)
```

Nested `for` loop

A for loop **inside another** for loop.Used for grids, tables, patterns, combinations, etc.

Example: 1

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

Example: 2

```
for i in range(3):        # Outer loop → rows
    for j in range(3):    # Inner loop → columns
        print("*", end=" ")
    print()
```

#print("*", end=" ") After printing *, don't go to a new line – just print a space and continue on the same line.

enumerate()

`enumerate()` gives **index + value** while looping.

Without `enumerate`:

```
fruits = ["apple", "banana", "mango"]
i = 0
for f in fruits:
    print(i, f)
    i += 1
```

With `enumerate`:

```
fruits = ["apple", "banana", "mango"]
for index, value in enumerate(fruits):
    print(index, value)
```

```
for index, value in enumerate(fruits, start=1):
    print(index, value)
```

List Comprehension

Short way to create a list using for (and optional if).

```
new_list = [expression for item in iterable]
```

Example 1: Squares of 1 to 5

```
squares = [x*x for x in range(1, 6)]
print(squares)   # [1, 4, 9, 16, 25]
```

Example 2: Only even numbers from 1 to 10

```
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)   # [2, 4, 6, 8, 10]
```

Example 3: Convert list of strings to uppercase

```
names = ["lx", "tom", "john"]
upper_names = [n.upper() for n in names]
print(upper_names)   # ['LX', 'TOM', 'JOHN']
```

Example: 4

```
students = [("Alice",75),("Tom",35),("john",60)]
pass_stud = [name.upper() for name,mark in students if mark>=40]
print(pass_stud)
```

What is a Function?

A **function** is a **block of code** that:

- Has a **name**
- Can be **reused** many times
- Can take **input (parameters)**
- Can **return output**

2. Types of Functions in Python

1. **Built-in Functions**
   → Already provided by Python. You just use them.

2. **User Defined Functions (UDF)**
   → You create them using `def`.

3. Built-in Functions

These are functions that come **ready-made with Python**.You don't define them; you just call them.

Function	Purpose	Example
`print()`	Show output	`print("Hello")`
`input()`	Take user input	`name = input()`
`len()`	Length of sequence	`len("Lx")`
`type()`	Data type	`type(10)`
`int()`, `float()`, `str()`	Type conversion	`int("10")`
`sum()`	Sum of items	`sum([1,2,3])`
`max()`, `min()`	Max / Min value	`max([4,9,2])`
`sorted()`	Sorted list	`sorted([3,1,2])`

`range()`	Number sequence	`range(5)`

```
nums = [10, 20, 30]

print("Length:", len(nums))       # 3

print("Sum:", sum(nums))         # 60

print("Max:", max(nums))         # 30

print("Sorted:", sorted(nums))   # [10, 20, 30]
```

4. User Defined Functions (UDF)

These are functions **you create** when built-in ones are not enough.

How to define a function?

```
def function_name(parameters):

    # body

    return value   # optional
```

Simple example – no parameters, no return

```
def greet():

    print("Hello, welcome to Python!")

greet()   # function call
```

Function with parameter(s) and no return

```
def greet_name(name):

    print("Hello", name)

greet_name("Lola")

greet_name("Rahul")
```

Function with parameters and return value

```
def add(a, b):
    result = a + b
    return result
sum1 = add(10, 20)
sum2 = add(5, 7)
print(sum1)  # 30
print(sum2)  # 12
```

Difference between print and return

print() is used to **show output to the user**.

```
def add(a, b):
    print(a + b)
add(5, 3)
result = add(5, 3)
print(result)
```

return is used to **send a value back from a function**.

```
def add(a, b):
    return a + b
result = add(5, 3)
print(result)
double = add(5, 3) * 2
print(double)
```

Real-life user-defined function example (Pass/Fail)

```
def check_result(mark):
    if mark >= 40:
        return "Pass"
    else:
        return "Fail"
print(check_result(75))   # Pass
print(check_result(30))   # Fail
```

Types of Arguments in Functions

1. Positional Arguments

Values are passed based on **position/order**.

Example 1

```
 def add(a, b):
    print(a + b)
add(10, 5)     # 10 -> a , 5 -> b
```

Example 2

```
def total_marks(tamil, english, maths):
    print("Total:", tamil + english + maths)

# Function call uses POSITION of values
total_marks(80, 70, 90)
```

2. Keyword Arguments

Specify the parameter names while calling the function.

Example 1

```
def student_info(name, age, course):
    print("Name:", name)
    print("Age:", age)
    print("Course:", course)
student_info(age=22, course="Data Analytics", name="Anna")
```

3. Default Arguments

Parameters get **default values** if arguments are not provided.

Example 1

```
def order_food(item, delivery="Normal"):
    print("Item :", item)
    print("Delivery Type :", delivery)

order_food("Pizza")
order_food("Burger", "Express")
```

4. Variable-Length Arguments — `*args`

When you don’t know how many arguments will be passed.

Example 1

```
def calculate_bill(*prices):
    return sum(prices)
```

```
print(calculate_bill(150, 230, 99))

print(calculate_bill(50, 75))
```

Example 2

```
def total_marks(*marks):

    print("Marks received:", marks)

    print("Total Marks:", sum(marks))


total_marks(80, 75, 90)

total_marks(65, 70, 85, 90, 88)
```

5. Variable-Length Arguments — **kwargs

`**kwargs` collects **any number of keyword arguments** .Data is stored as a **dictionary**

Example 1

```
def student_register(**details):
    print("Student Details:")
    for key, value in details.items():
        print(key, ":", value)

student_register(name="Ann", age=22, course="Data Science")

student_register(name="Rahul", course="AI", city="Kochi",
phone="9876543210")
```

Example 2

```
def place_order(**order):
    print("Order Details:")
    print(order)

place_order(item="Pizza", size="Medium", cheese="Extra",
delivery="Express")
```

Local and Global Namespace

1. Local Namespace

Variables declared **inside a function** → accessible **ONLY inside that function**

```
def test():
    x = 10          # Local variable
    print(x)

test()
# print(x)   #Error
```

2. Global Namespace

Variables declared **outside functions** → accessible **everywhere**

```
y = 20           # Global variable

def test():
    print(y)

test()
print(y)
```

3. Using `global` keyword

Use when you want to **change a global variable inside a function**.

```
count = 0

def increment():
    global count
    count += 1

increment()
increment()
print(count)     # Output: 2
```

Introduction to oops in python

OOPS = **Object Oriented Programming System**

OOPS is a programming approach where a program is designed using **objects** that represent real-world entities, and these objects are created from **classes**, which act as blueprints.

1. Class – Definition

A **class** is a **blueprint or template** used to create objects.
It describes **what an object will have (attributes)** and **what it can do (methods)**.

Simple Definition:

> A **class** is a design of a real-world object.

```
class Car:

    pass
```

Here, Car is just a design — no real car yet.

2. Object – Definition

An **object** is a **real instance of a class**.

Simple Definition:

> An **object** is a **created item from a class** — a real version of the design.

Example:

```
car1 = Car()
```

Now `car1` is a **real object**.

3. Attributes – Definition

Attributes are the **data or properties** of an object.

Simple Definition:

Attributes tell what an object has.

Real-life Car attributes:

- color
- brand
- speed

```
class Car:

    def __init__(self, brand, color):

        self.brand = brand

        self.color = color
```

- `brand` and `color` are **attributes**

What is `__init__` and `self` in Python?

`__init__` is a special method (constructor) that runs automatically whenever an object is created from a class.

`__init__` is used to initialize (assign values to) the attributes of an object when the object is created.

`self` represents the current object.

`self` is used to access attributes and methods of the current object inside the class.

4. Methods – Definition

Methods are the **functions defined inside a class**.

Simple Definition:

Methods tell what an object can do.

Real-life Car methods:

- start()

- drive()
- stop()

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
    def start(self):
        print("Car started")
    def show_details(self):
        print("Brand:", self.brand)
        print("Color:", self.color)
```

Full Example :

```
class Car:

    def __init__(self, brand, color):
        self.brand = brand        # Attributes
        self.color = color

    def start(self):              # Method
        print("Car started")

    def show_details(self):       # Method
        print("Brand:", self.brand)
```

```
        print("Color:", self.color)


# Creating objects
car1 = Car("Tata", "Red")
car2 = Car("Hyundai", "White")


# Using methods
car1.start()
car1.show_details()


car2.start()
car2.show_details()
```

OOPS Concepts – Definition with Example

---

1. Class

Definition:
A class is a *blueprint or design* used to create objects.

Real-life example:
Blueprint of a car (design with color, speed, start())

---

2. Object

Definition:
An object is a *real instance of a class*.

Real-life example:
A real car on the road made from the car design

---

3. Encapsulation

Definition:
Encapsulation is *binding data and methods together into one unit (class)*.

Real-life example:
A mobile phone — contacts + calling functions packed in one device

---

4. Inheritance

Definition:
Inheritance is when a *child class gets properties and methods of its parent class*.

Real-life example:
Son inherits surname and property from father

---

5. Polymorphism

Definition:
Polymorphism means *one method performs different actions for different objects*.

Real-life example:
Sound — Dog barks, Cat meows, Cow moos

---

6. Abstraction

Definition:
Abstraction means *hiding complex internal workings and showing only essential features*.

Real-life example:
Driving a car without knowing engine working